Programování 2

5. cvičení, 17-3-2026

Farní oznamy

1. Tento text a kódy ke cvičení najdete v repozitáří cvičení na https://github.com/PKvasnick/Programovani-2.
2. **Domácí úkoly**: 2 nové úkoly od minulého cvičení.
    1. Permutace řádků matice
    2. Bucket sort pro seznam slov

    Vrátíme se k nim jenom z pedagogických důvodů.

Dnešní program:

- Kvíz
- Python: jak rychlý je můj kód
- Pořád třídění

Na zahřátí

> *"Before software can be reusable it first has to be usable." – Ralph Johnson*

Opakovaná použitelnost kódu se přeceňuje. Pokud se stanete programátory, budete samozřejmě psát produkční kód s vysokými požadavky na udržitelnost, čistotu kódu a dokumentaci. Ale nejčastěji využijete nevelké, rychle a špinavě napsané kousky kódu. Velké knihovny sami po sobě dědíme zřídka.

Co dělá tento kód

```
x = 5
y = 7

x = x + y
y = x - y
x = x - y

print(f"{x=} {y=}")
```

Co bude rychlejší? Pythonské `x, y = y, x`, anebo série třech sčítání/odečítání?

Zjistíte v následující části.

Jak rychlý je můj kód?

Často se v tomto semestru zabýváme tím, jak je který algoritmus nebo kód rychlý. Jak ale můžeme změřit rychlost kódu?

Příklad: Ciferný součet - tři metody

```
def cif_soucet_1(cislo: int) -> int:
    """mod 10 dává číslici, div 10 zbytek čísla"""
    soucet = 0
    while cislo:
        soucet += cislo % 10
        cislo //= 10
    return soucet

def cif_soucet_2(cislo: int) -> int:
    """Převedeme číslo na string, rozdělíme na číslice a ty sečteme."""
    return sum(map(int, str(cislo)))

def cif_soucet_3(cislo: int) -> int:
    """divmod 10 dává dvojici (zbyvajici_cislo, číslice)"""
    soucet = 0
    while cislo:
        cislo, cislice = divmod(cislo, 10)
        soucet += cislice
    return soucet
```

Který kód je rychlejší?

```
# soubor speedy.py v adresáři code/Ex05
from timeit import Timer

def cif_soucet_1(cislo: int) -> int:
    """mod 10 dává číslici, div 10 zbytek čísla"""
    soucet = 0
    while cislo:
        soucet += cislo % 10
        cislo //= 10
    return soucet

def cif_soucet_2(cislo: int) -> int:
    """Převedeme číslo na string, rozdělíme na číslice a ty sečteme."""
    return sum(map(int, str(cislo)))

def cif_soucet_3(cislo: int) -> int:
    """divmod 10 dává dvojici (zbyvajici_cislo, číslice)"""
    soucet = 0
    while cislo:
        cislo, cislice = divmod(cislo, 10)
```

```
        soucet += cislice
    return soucet

if __name__ == "__main__":

    t1 = Timer("cif_soucet_1(123456789)", "from speedy import cif_soucet_1")
    print(1, t1.timeit(number = 1000000))

    t2 = Timer("cif_soucet_2(123456789)", "from speedy import cif_soucet_2")
    print(2, t2.timeit(number = 1000000))

    t3 = Timer("cif_soucet_3(123456789)", "from speedy import cif_soucet_3")
    print(3, t3.timeit(number = 1000000))

------
1 0.2282492001540959
2 0.33217189996503294
3 0.30377340014092624
```

Všimněte si, že volání časovače není úplně pohodlné.

Modul `timeit` vlastně spouští nový shell, který spustí požadovaný kód v operačním systému (je to proto, že `timeit` modifikuje některá nastavení Pythonu (např. vypíná *garbage collector*, tedy mechanizmus, který odstraňuje z paměti objekty, které se dostali mimo jmenný prostor, kde byly vytvořeny). Proto musíte zadat nejen název funkce, kterou chcete spustit, ale také ukázat, kde ji Python najde (`from speedy import cif_soucet_1` atd.).

Pohodlnější je použít *magic* `%%timeit` v IPythonu, tedy např.

- v Pythonské konzoli PyCharmu
- v jakémkoli typu Jupyter notebooku - Google Colab, JupyterLab a pod.

Pusťte si měření víckrát.

Všimněte si, že měření času *není stabilní*: potřebujeme velkou statistiku, abychom získali jakž-takž konzistentní představu o relativní rychlosti kódů. Je to proto, že měření běží v multitaskovém prostředí a nikdy nevíme, kdy operační systém vlákno s testy odloží bokem a začne si vyřizovat něco co právě považuje za důležitější.

Který kód je rychlejší?

Nejpomalejší kód je ten, který vypadá nejmoudřeji - funkce `cif_soucet_2` . Používá `sum` a `map` a žádný otevřený cyklus, jenomže dělá konverzi z `int` na `str` a zpátky, a to je složitá operace. Na druhé straně vlastně vidíme, že použít tento rychlý trik k nalezení ciferního součtu čísla není tragicky pomalejší než ostatní metody.

Nejrychlejší kód je, zdá se, `cif_soucet_1`, nejspíš proto, že ho kompilátor dokáže dobře zoptimalizovat.

`cif_soucet_3` je znatelně pomalejší, nejspíš proto, že vytváří a rozbaluje tuply.

Čtení z konzole

Jedna varianta, kterou jsme dosud neměli:

Nežádej o povolení, ale o prominutí

```
from contextlib import suppress

with suppress(EOFError):
    while p := input():
        print(int(p))
```

1. Když už není co číst, `input()` vyvolá výjimku. Takže výjimku zachytíme a klidně pokračujeme dál. Případný řádek s ukončovacím řetězcem lehce odstraníme. Takovýto postup se v Pythonu docela často používá, i když bychom měli výjimky používat nejvíc pro ošetření stavů, které nejsou pod naši kontrolou.
2. Pro zachycení výjimky nepoužíváme blok `try-except`, ale *context manager* `suppress`, tedy prostředí, které umí vrátit věci do pořádku, když se v bloku kódu stane něco ošklivého. Context managery nejspíš znáte jako prostředí, kde komunikujeme s textovým souborem, ale jejich použití je mnohem širší.
3. Toto je celkem dobrý nástroj pro načítání dat, které nemají ukončovací řetězec. V porovnání s `try-except` vyžaduje (malou) režii pro zřízení třídy kontextového manažera, ale na druhé straně jasněji deklaruje úmysl: "V následujícím bloku může dojít k této chybě.".

Třídění

Abychom mohli věci třídit, musí tyto věci implementovat operátory srovnání.

Selection sort

Zleva nahrazujeme hodnoty minimem zbývající části posloupnosti:

```
def selection_sort(b):
    for i in range(len(b) -1):
        j = b.index(min(b[i:]))
        b[i], b[j] = b[j], b[i]
    return b
```

Proč tento kód funguje? Tedy: jak vlastně dokazujeme správnost algoritmu? Třeba tohoto?

$O(n^2)$ operací, $O(1)$ paměť.

Bubble sort

```
def bubble_sort(b):
    for i in range(len(b)):
        n_swaps = 0
        for j in range(len(b)-i-1):
            if b[j] > b[j+1]:
                b[j], b[j+1] = b[j+1], b[j]
                n_swaps += 1
        if n_swaps == 0:
            break
    return b
```

Toto vypadá, že je jen další ne příliš efektivní algoritmus: $O(n^2)$ operací, $O(1)$ paměť. Ale co když mám datovou strukturu, kterou můžu procházet jen jedním směrem?

Insertion sort

Začínáme třídit z kraje seznamu, následující číslo vždy zařadíme na správné místo do již utříděné části.

```
def insertion_sort(b):
    for i in range(1, len(b)):
        up = b[i]
        j = i - 1
        while j >= 0 and b[j] > up:
            b[j + 1] = b[j]
            j -= 1
        b[j + 1] = up
    return b
```

$O(n^2)$ operací, $O(1)$ paměť. Typicky se používá pro nalezení mediánu mediánů, viz dále.

Counting sort

Pro celá čísla nebo podobné objekty s omezeným rozsahem hodnot máme přirozenou metodu třídění v lineárním čase:

- pro každou možnou hodnotu objektu si zřídíme příhrádku
- roztřídíme data podle přihrádek
- sesypeme přihrádky do výsledného pole. Pomůžeme si kumulativními součty obsazeností přihrádek, které nám dají pozice pro jednotlivé hodnoty ve výsledném setříděném poli. Postupujeme od konce pole a "vyprázdňujeme" přihrádky do správných pozic. Toto zaručí, že třídění bude stabilní - tedy stejné prvky budou ve stejném pořadí jako ve vstupním poli.

Implementace může vypadat takto:

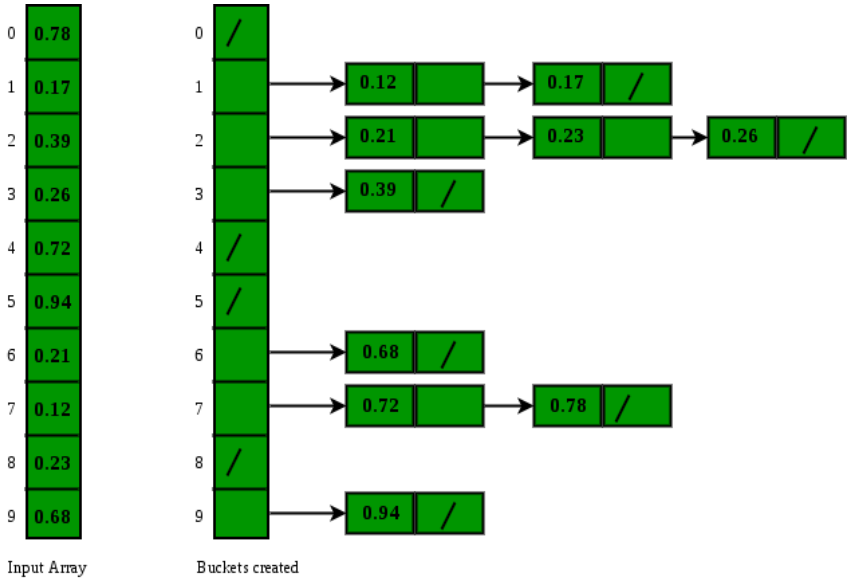
```
from random import randint

def count_sort(b:list[int], rmax: int) -> list[int]:
    bins = [0] * rmax   # vytvoříme přihrádky
```

```
    # roztřídíme
    for elem in b:
        bins[elem] += 1
    # kumulativní součty obsazeností
    for i in range(1,rmax):
        bins[i] += bins[i-1]
    # naplníme výsledné pole
    output = [0] * len(b)
    for i in reversed(range(len(b))):
        j = b[i]
        bins[j] -= 1
        output[bins[j]] = j
    return output

r = 10
data = [randint(0,r-1) for _ in range(100)]

print(*data)
print(*count_sort(data, r))
```

Kritická hodnota u tohoto algoritmu je počet možných hodnot ve vstupním poli. Pokud tuto hodnotu známe a je podstatně menší než počet hodnot ve vstupním poli, je třídění lehké. Nezapomínejme také na to, že pokud možné hodnoty nejsou čísly, budeme je možná muset setřídit.

Bucket sort / Radix sort

- Nahrubo si setřídíme čísla do přihrádek
- Setřídíme obsah přihrádek
- Spojíme do výsledného seznamu



- Jednoduchý algoritmus
- Občas musíme popřemýšlet, jak vytvořit přihrádky (viz domácí úkol)
- V závislosti od charakteru dat můžeme také použít counting sort
- Třídíme od konce, abychom se vyhnuli rekurzi.

- Můžeme také třídit od začátku, ale takové řešení je rekurzivní a má tedy podstatně větší režii a možná omezení pro velká data.

Rekurzivní řešení

Toto bylo k vidění u mnohých z vás, a je to přirozený a docela elegantní způsob, jak napsat kód pro tuto úlohu. Typické řešení vypadalo takto:

```
def get_bucket(word: str, letter: int) -> int:
    return ord(word[letter]) - 96 if letter < len(word) else 0


def to_buckets(words: list[str], letter: int) -> list[int]:
    if len(words) < 2:  # není co třídit
        return words
    buckets = [[] for _ in range(27)]
    for word in words:
        buckets[get_bucket(word, letter)].append(word)
    sorted_words = []
    for bucket in buckets:
        sorted_words.extend(to_buckets(bucket, letter + 1))
    return sorted_words


def main():
    words = []
    while True:
        word = input().strip()
        if word == '-end-':
            break
        words.append(word)
    result = to_buckets(words, 0)
    for word in result:
        print(word)
    print("-end-")


if __name__ == '__main__':
    main()
```

Nerekurzivní řešení

Namísto toho, abychom pokaždé sesypali kyblíky do pole, prostě průběžně dělíme věci do z kyblíků do jiných kyblíků. Potřebujeme ale zarážky, abychom mohli oddělit data předešlého a aktuálního třídícího cyklu. A potřebujeme kontejner, který umí levně přidávat a ubírat hodnoty na obou koncích.

collections.deque - pole optimalizované pro přídávání a odebírání prvků na začátku a na konci.

https://docs.python.org/3/library/collections.html#collections.deque

:marks Menu | Evidence fyzikálního p... | Jednoznakovky | Python | Programování 2 | Píchačky | Contexto | Phind: AI | DoucMa.sk

:atypes

d Recipes

ction for

and

-related

Base Classes

deque objects

class collections.**deque**([*iterable*[, *maxlen*]])

Returns a new deque object initialized left-to-right (using append()) with data from *iterable*. If *iterable* is not specified, the new deque is empty.

Deques are a generalization of stacks and queues (the name is pronounced "deck" and is short for "double-ended queue"). Deques support thread-safe, memory efficient appends and pops from either side of the deque with approximately the same *O*(1) performance in either direction.

Though list objects support similar operations, they are optimized for fast fixed-length operations and incur *O*(*n*) memory movement costs for `pop(0)` and `insert(0, v)` operations which change both the size and position of the underlying data representation.

If *maxlen* is not specified or is `None`, deques may grow to an arbitrary length. Otherwise, the deque is bounded to the specified maximum length. Once a bounded length deque is full, when new items are added, a corresponding number of items are discarded from the opposite end. Bounded length deques provide functionality similar to the `tail` filter in Unix. They are also useful for tracking transactions and other pools of data where only the most recent activity is of interest.

Deque objects support the following methods:

append(*x*)

Add *x* to the right side of the deque.

appendleft(*x*)

Add *x* to the left side of the deque.

Zatímco seznam má položky uložené v paměti bezprostředně vedle sebe (což umožňuje rychlý přístup přes index), `deque` si můžete představit jako obousměrně spojovaný seznam - tedy položky, uložené nezávisle v paměti a vzájemně propojené pointry.

Nejdůležitější metody: `pop`, `popleft`, `append`, `appendleft`.

Operace na levém konci: O(1), pro seznam to je O(n) a některé vůbec neumí.

Operace na pravém konci: O(1) stejně jako pro seznam (ve skutečnosti toto umí seznam o něco málo rychleji).

Operace uprostřed: `pop(index)` má složitost O(n) pro seznam, `deque` musí udělat `rotate - pop - rotate`, ale stejně to udělá rychleji než seznam.

`collections.deque` je ideální kontejner pro implementaci front a zásobníků.

```
import string
from collections import deque

bucket_labels = ' ' + string.ascii_lowercase

class Distributor:
    """Translates k-th letter to bucket number"""
    def __init__(self):
        self.dictionary = {j: i for i, j in enumerate(bucket_labels)}

    def get_bucket(self, word: str, k: int):
```

```python
        return self.dictionary[word[k]] if len(word) > k else 0


terminator = "" # signals end of bucket content
buckets = {i : deque() for i in range(len(bucket_labels))}

def cap_buckets():
    for d in buckets.values():
        d.append(terminator)


dist = Distributor()

def sort_cycle(k: int):
    for d in buckets.values():
        while True:
            word = d.popleft()
            if word == terminator:
                break
            new_bucket = dist.get_bucket(word, k)
            buckets[new_bucket].append(word)
    cap_buckets()


def extract_bucket(k: int):
    for d in buckets.values():
        while True:
            word = d.popleft()
            if word == terminator:
                break
            print(word)
    print("-end-")


def main():
    while True:
        word = input().strip()
        if word == "-end-":
            break
        buckets[0].append(word)
    k = len(max(buckets[0], key = len))
    cap_buckets()
    for k in range(k-1, -1, -1):
        sort_cycle(k)
    extract_bucket(k)


if __name__ == "__main__":
    main()
```

Quickselect, quicksort

Obě metody sérii částečných třídění podle pivotu, při kterých se seznam dělí na hodnoty menší než pivot, rovné pivotu a větší než pivot. Jediný rozdíl je v tom, že QuickSelect hledá k-tou největší hodnotu, a proto po každém dělení pokračuje tou částí původního seznamu, kde se tato hodnota určitě nachází. QuickSort třídí celé pole a tudíž se rekurzivně volá na všechny tři části.

Symbolicky, kód pro QuickSort funguje takto:

```
from random import randint

def quick_sort(b):
    if len(b) < 2:
        return b
    if len(b) == 2:
        return [min(b), max(b)]
    pivot = b[randint(0, len(b)-1)]
    lows = [x for x in b if x < pivot]
    pivots = [x for x in b if x == pivot]
    highs = [x for x in b if x > pivot]
    return quick_sort(lows) + pivots + quick_sort(highs)


data = [randint(1,100) for _ in range(10)]

print(data)
print(quick_sort(data))
```

Proč symbolicky?

- Pro každou úroveň volání vytvářime další kopii původního seznamu.
- Provádíme mnoho manipulací s poli
- Rekurze není zdarma a může narazit na omezení Pythonu.

Tedy máme poměrně hezký kód, který je určitě funkční, ale není zralý pro nasazení.

Dvě hlavní věci, které můžeme zlepšit, jsou volba pivotu a přerovnání dat při uspořádání podle pivotu.

Volba pivotu: median-of-medians

Optimální pivot je medián, tedy hodnota, od které je 50% ostatních hodnot menších a 50% větších. Jenomže výpočet mediánu je úloha ekvivalentní tomu, co řešíme (nalézt n//2-tou největší hodnotu), Běžný postup je hierarchicky spočíst mediány pětic čísel (např. pomocí insertion sort, až dospějeme k rozumnému odhadu pivotu.

Přerovnání dat

Jak podle vás funguje tento kód?

Udělejte si z toho cvičení v porozumění kódu. Pomůže vám tužka a papír. Všimněte si, jak a jestli vám ve čtení pomáhají komentáře.

Jaká je složitost tohoto kódu?

```python
def partition(data, left, right, pivotIndex, n):
    pivotValue = data[pivotIndex]
    data[pivotIndex], data[right] = data[right], data[pivotIndex]  # Move pivot to end
    storeIndex = left
    # Move all elements smaller than the pivot to the left of the pivot
    for i in range(left, right):
        if data[i] < pivotValue:
            data[storeIndex], data[i] = data[i], data[storeIndex]
            storeIndex += 1
    # Move all elements equal to the pivot right after
    # the smaller elements
    storeIndexEq = storeIndex
    for i in range(storeIndex, right):
        if data[i] == pivotValue:
            data[storeIndexEq], data[i] = data[i], data[storeIndexEq]
            storeIndexEq += 1
    data[right], data[storeIndexEq] = data[storeIndexEq], data[right]  # Move pivot to its
final place
    # Return location of pivot considering the desired location n
    if n < storeIndex:
        return storeIndex  # n is in the group of smaller elements
    if n <= storeIndexEq:
        return n  # n is in the group equal to pivot
    return storeIndexEq  # n is in the group of larger elements

data = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
print(data)
pivotIndex = 2  # 3rd element is 4
n = 6        # 6th smallest element is 6
result = partition(data, 0, len(data)-1, pivotIndex, n)
print("Partitioned index:", result)
print("Data after partitioning:", data)
```

> **Note**
>
> Můžeme nahlížet na bucket sort jako na zobecněné dělení dat podle pivotu: namísto jednoho pivotu jich máme několik a tedy v každém kroku získáváme např. lepší odhad k-té největší hodnoty.

Domácí úkoly

Dvé úlohy inspirované reálným světem. Snažte se najít vhodné datové struktury pro reprezentaci dat obou úloh, abyste získali co nejhezčí a nejefektivnější kód.

1. **Parkoviště** - Na parkoviště přicházejí auta, pokud je plné, čekají, než se najde místo, zaparkují a později odcházejí. Máte vypočíst celkovou tržbu parkoviště za 1 den.
2. **Maximální počet hostů v restauraci:** do restaurace v nepravidelných okamžicích přicházejí hosté a po k hodinách odcházejí. Máte zjistit, kdy je v restauraci maximální počet hostů a počítají se pouze průběžná řešení.